Universidad de Ingeniería y Tecnología (UTEC)
Escuela de Ciencia de la Computación

Curso: Base de Datos 2, ciclo 2026-2

Minigestor de base de datos multimodal

Informe del proyecto: Parte 1 (base de datos relacional) y Parte 2 (base de datos espacial)

Integrantes

Luis Izaguirre
Dayane Rojas
Noe Paredes

Lima, 4 de octubre de 2026

Repositorio: `https://github.com/LuisIZ/BD2_Proyecto_2026-2`

Índice

1. Introducción **3**

2. Arquitectura del sistema **3**

2.1. Organización del código 4

3. Dominio de datos **4**

3.1. Datasets 4

3.2. Tipos de datos 4

3.3. Formato de registro y de página 5

3.4. Dataset de productos para las Partes 3 y 4 5

4. Parte 1: técnicas implementadas **5**

4.1. Heap File con páginas slotted 5

4.2. Archivo secuencial paginado 6

4.3. B+ agrupado 6

4.4. B+ no agrupado 6

4.5. Hash extensible en disco 6

4.6. Algoritmos externos 7

4.7. Parser, planificador y EXPLAIN 7

4.8. Transacciones y concurrencia 7

4.9. Interfaz de usuario 8

5. Parte 2: base de datos espacial **8**

5.1. Métricas de distancia 8

5.2. Extensión SQL 8

5.3. Índice R-Tree 9

5.4. Intersección con polígono 9

5.5. Estado de la Parte 2 10

6. Parte 4: búsqueda por similitud de imágenes **10**

6.1. Índice IVF 10

7. Interfaz para las Partes 3 y 4 **11**

8. Comparación experimental **11**

8.1. Metodología 11

8.2. Heap File frente a archivo secuencial 12

8.3. Índices: B+ agrupado, B+ no agrupado y hash extensible 12

8.4. B+ agrupado frente a B+ no agrupado a través de SQL 16
8.5. Transacciones 16
8.6. Parte 2: R-Tree frente a búsqueda secuencial 16
8.7. Parte 2: secuencial, R-Tree y GiST 17
8.8. Parte 3: TF-IDF, BM25 y GIN 19
8.9. Parte 4: k-NN secuencial, IVF y HNSW 22
8.10. Cuándo usar cada técnica 29

9. Conclusiones y límites conocidos **29**

1. Introducción

El proyecto consiste en construir desde cero un gestor de bases de datos que soporte varios tipos de datos. Este informe cubre las dos primeras partes del enunciado. La Parte 1 corresponde a una base de datos relacional con tablas y SQL: organización de archivos en disco, índices, algoritmos externos, procesamiento de consultas, transacciones y una interfaz gráfica. La Parte 2 extiende el motor con datos espaciales: coordenadas geográficas, métricas de distancia y consultas por radio y por cercanía.

El objetivo del curso no es competir con un motor comercial, sino entender cómo funciona por dentro. Por eso el equipo escribió cada estructura sobre páginas de 4 KB y midió cuántas páginas lee y escribe cada operación. Esa medida, más que el tiempo, es la que permite explicar por qué una técnica conviene más que otra.

El informe sigue la estructura pedida en la sección de entregables: primero la arquitectura y el dominio de datos, luego la explicación de cada técnica implementada y al final la comparación experimental. La última sección resume lo que quedó pendiente. El código fuente, la documentación técnica de cada módulo y este informe están en el repositorio del proyecto: `https://github.com/LuisIZ/BD2_Proyecto_2026-2`.

2. Arquitectura del sistema

El sistema está dividido en capas. Cada capa solo conoce a la que tiene debajo, lo que permitió que distintos integrantes trabajaran en paralelo sobre interfaces acordadas.

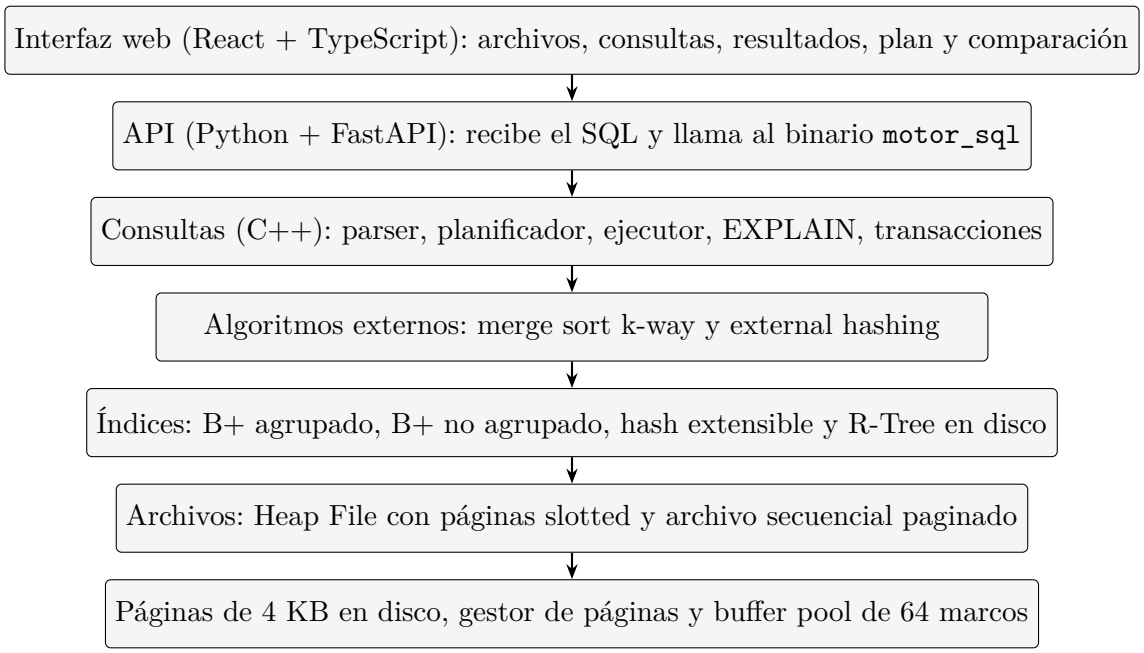


Figura 1: Capas del minigestor.

El motor está escrito en C++17 y se compila con un `Makefile`. Su punto de entrada es el binario `motor_sql`, que recibe un lote de sentencias SQL y responde en JSON con las filas, el plan de ejecución y las páginas leídas. La API en Python solo traduce peticiones HTTP a llamadas a ese binario, y la interfaz web dibuja las respuestas. Así, todo lo que toca el disco está en C++ y puede probarse sin la interfaz.

2.1. Organización del código

```
motor/
  comun/          interfaz IFileOrganization y Registro, compartidos
  archivos/       heap_file, sequential_file, pagina_slotted
  indices/        bplus_agrupado, bplus_no_agrupado, hash_extensible_disco,
                  rtree, gestor_paginas, buffer_pool
  consultas/      parser_sql, catalogo, ejecutor, external_algorithms, motor_sql
  espacial/       distancia.h (métricas euclidiana y haversine)
  transacciones/  gestor_locks.h (2PL estricto y deadlocks)
  pruebas/        una prueba por estructura y los benchmarks
api/              API en Python que llama al motor
web/              interfaz React
datos/            datasets CSV y resultados de los experimentos
docs/             documentación técnica de cada módulo y este informe
```

Cada módulo tiene su prueba en `motor/pruebas/`, y `make test` las ejecuta todas. Al momento de entregar, los diez programas de prueba terminan sin errores.

3. Dominio de datos

3.1. Datasets

El conjunto principal es `organizations`, un CSV público con datos de empresas en tres tamaños: 1 000, 10 000 y 100 000 filas. Sus columnas son `Index`, `Organization Id`, `Name`, `Website`, `Country`, `Description`, `Founded`, `Industry` y `Number of employees`. La columna `Index` es un entero único y sirve como clave primaria; `Founded` y `Number of employees` son enteros con muchos valores repetidos, útiles para probar índices secundarios. La fila promedio mide unos 140 bytes y la más larga 220 bytes.

Para la Parte 2 se usan puntos de Lima (centro, San Isidro, Miraflores, Barranco y Callao) en las pruebas de distancia, y un archivo pequeño de alumnos sirve para probar tablas creadas a mano.

3.2. Tipos de datos

El motor maneja tres tipos de columna:

- `INT`: entero de 32 bits.
- `VARCHAR(n)`: texto de hasta n bytes.
- `POINT`: coordenada geográfica (latitud, longitud). Se guarda como dos enteros de 32 bits en microgrados, es decir, grados multiplicados por un millón. Ocupa 8 bytes y tiene una precisión de unos 11 cm.

La clave primaria debe ser `INT` y puede estar en cualquier columna. Los decimales de las coordenadas se convierten a microgrados leyendo las cifras como texto, sin pasar por números de punto flotante, para que `POINT(-12.0464, -77.0428)` se guarde exactamente como `-12046400` y `-77042800`.

3.3. Formato de registro y de página

Todas las estructuras trabajan sobre páginas de 4 096 bytes. En el Heap File y en el archivo secuencial, un registro es `[int32 clave][uint32 largo][valor]`, donde el valor empaqueta el resto de columnas: los enteros en 4 bytes y los textos con su largo en 2 bytes seguido de los caracteres. El B+ agrupado usa registros de tamaño fijo, con la clave primero y cada texto rellenado hasta su largo máximo, para poder ubicarlos por posición dentro de la hoja.

3.4. Dataset de productos para las Partes 3 y 4

Las Partes 3 y 4 usan un mismo dataset: *Amazon Berkeley Objects* (ABO), publicado por Amazon bajo la licencia CC BY 4.0. Cada producto tiene texto (nombre, viñetas y descripción) y una foto de catálogo, y como el texto y la imagen comparten el mismo `id`, una consulta puede combinar la búsqueda de texto con la de imágenes.

ABO trae 147 702 *listings*, porque el mismo producto aparece una vez por cada marketplace de Amazon. El script `tools/dataset/preparar.py` se queda con un registro por producto, solo con texto en inglés, quita el HTML y los saltos de línea, y recorta la descripción a 1 000 bytes. Quedan 121 549 productos con imagen y nombre en inglés, de los que se toman tres subconjuntos anidados de 1 000, 10 000 y 100 000 filas con la semilla 2026. Los primeros 1 000 productos de cualquiera de los tres archivos son los mismos, así que un resultado sobre 1k se puede repetir sobre 10k.

```
CREATE TABLE productos (id INT PRIMARY KEY, item_id VARCHAR(20), nombre VARCHAR(200),
  tipo VARCHAR(50), categoria VARCHAR(200), marca VARCHAR(80), color VARCHAR(40),
  descripcion VARCHAR(1000), imagen VARCHAR(80));
COPY productos FROM FILE 'datos/productos_10k.csv';
```

La columna `descripcion` es la que se indexa en la Parte 3, e `imagen` es la ruta de la foto principal (lado mayor de 256 px) que se usa en la Parte 4. Las columnas `tipo` y `categoria` no se indexan y sirven como juicio de relevancia en los experimentos. Los textos se recortan en bytes UTF-8 sin partir caracteres, porque el motor mide el largo en bytes, y así un registro ocupa como mucho unos 1,7 KB y cabe en una página. El `COPY` de los 100 000 productos tarda alrededor de un segundo.

El dataset está muy cargado hacia un tipo: 53 355 de los 100 000 productos son fundas de celular (`CELLULAR_PHONE_CASE`), seguidas de zapatos (8 027) y abarrotes (5 044). Además, 15 571 productos comparten la foto principal con otro, lo que produce vectores idénticos y empates en la búsqueda por similitud. Los detalles de la limpieza están en `docs/dataset.md`.

4. Parte 1: técnicas implementadas

4.1. Heap File con páginas slotted

El Heap File guarda los registros en orden de llegada. Cada página tiene una cabecera con el número de slots y el inicio del espacio libre, un directorio de slots que crece hacia abajo y los datos que crecen desde el final hacia arriba. Un registro se identifica por su RID, el par (página, slot), y ese RID no cambia aunque otros registros se borren, porque borrar solo marca el slot como tumba.

Para reutilizar espacio, el archivo mantiene en memoria un mapa del espacio libre de cada página, que se reconstruye al abrir leyendo solo las cabeceras. Así una inserción no recorre el archivo

buscando sitio: consulta el mapa, lee una página y la escribe. La prueba del enunciado (insertar 1 000, borrar 500 e insertar 500) termina sin que el archivo crezca.

La desventaja es que el heap no tiene orden, así que buscar por clave primaria sin índice obliga a recorrer el archivo.

4.2. Archivo secuencial paginado

El archivo secuencial mantiene los registros ordenados por clave dentro de cada página y entre páginas. Tiene un área principal ordenada y un área auxiliar donde van los registros que no caben en su página.

- La búsqueda hace búsqueda binaria sobre las páginas principales leyendo solo la primera clave de cada una, luego búsqueda binaria dentro de la página y, si no encuentra la clave, recorre el auxiliar.
- La inserción ubica la página por búsqueda binaria; si hay espacio, inserta en su posición ordenada, y si no, el registro va al área auxiliar.
- La eliminación es perezosa: marca el slot como tumba y lo deja en su sitio, porque su clave todavía sirve como frontera para la búsqueda binaria.
- La reorganización se dispara cuando las tumbas y los registros auxiliares superan el 30 % del total. Hace una mezcla en streaming del área principal con el auxiliar ordenado y deja las páginas al 80 % para que las siguientes inserciones entren en su lugar.

4.3. B+ agrupado

En el B+ agrupado las hojas del árbol contienen las filas completas, ordenadas por clave primaria, y están encadenadas. Una búsqueda baja de la raíz a la hoja leyendo una página por nivel, y un rango recorre hojas consecutivas. Con filas de 180 bytes caben 22 registros por hoja. El árbol implementa split de hojas y nodos internos con propagación hasta la raíz, eliminación con préstamo y fusión, y una carga masiva que construye las hojas al 90 % de su capacidad. Solo puede haber uno por tabla, porque define el orden físico de los datos.

4.4. B+ no agrupado

El B+ no agrupado es un índice secundario: sus hojas guardan pares (clave, RID) que apuntan al Heap File, con 16 bytes por entrada y unas 205 entradas por página. Admite claves repetidas y puede crearse sobre cualquier columna `INT` con `CREATE INDEX`. Su costo está en que cada fila encontrada obliga a leer una página del heap, y esas páginas están dispersas.

4.5. Hash extensible en disco

El hash extensible mantiene un directorio de 2^d punteros a buckets, donde d es la profundidad global. Cada bucket es una página de 4 KB con 255 entradas (clave, RID). Cuando un bucket se llena y sus entradas pueden separarse por un bit más del hash, se divide, y si su profundidad local ya era igual a la global, el directorio se duplica. Si todas las entradas tienen la misma clave, dividir no sirve, y en ese caso se encadena una página de desborde. La profundidad máxima es un parámetro del constructor y por defecto vale 16 bits (65 536 entradas de directorio), un valor

que el profesor sugirió revisar en el primer avance. Al llegar a ese límite el directorio deja de duplicarse y los buckets llenos pasan a usar páginas de desborde.

El hash responde una igualdad con una lectura del directorio y una del bucket, pero no tiene noción de orden, así que el planificador nunca lo usa para rangos.

4.6. Algoritmos externos

External merge sort. Para `ORDER BY`, los registros se dividen en corridas del tamaño del presupuesto de memoria (páginas de buffer por registros por página). Cada corrida se ordena y luego se mezclan k a la vez con un heap de mínimos, en $O(N \log k)$. La traza del plan reporta las corridas iniciales y las pasadas de mezcla.

External hashing para GROUP BY. Las filas se reparten en particiones según el hash de la columna de agrupación. Como todas las filas de un grupo caen en la misma partición, cada partición se agrega por separado con su propia tabla de hash y luego se libera. Así en memoria solo viven los grupos de la partición en curso: con 300 grupos repartidos en 9 particiones, el máximo de grupos simultáneos fue 42. El plan muestra ese valor como `grupos_en_memoria`.

4.7. Parser, planificador y EXPLAIN

El parser es de descenso recursivo y reconoce `CREATE TABLE` (con esquema o desde un CSV), `COPY`, `CREATE INDEX ... USING BPLUS | HASH`, `INSERT`, `DELETE`, `SELECT` con `WHERE`, `GROUP BY`, `ORDER BY` y `LIMIT`, `DROP`, `SHOW TABLES`, `DESCRIBE`, `EXPLAIN [ANALYZE]` y las sentencias de transacción.

El planificador elige una sola condición del `WHERE` que pueda resolverse con una estructura (índice, clave del agrupado o búsqueda binaria del secuencial) y deja el resto para un filtro en memoria. La decisión se toma mirando solo el catálogo, sin leer datos, y por eso `EXPLAIN` puede mostrar el plan sin ejecutar la consulta. `EXPLAIN ANALYZE` sí la ejecuta y agrega a cada nodo las filas, páginas y milisegundos reales, y termina con el tiempo de planificación (que incluye el parseo) y el de ejecución, como en PostgreSQL:

```
Limit on t  (rows=3) (actual rows=3)
  -> Projection on t (actual rows=15)
    -> Sort on t (actual time=1.151 rows=15)
       Sort Key: Index ASC
      -> Index Scan using idx_f on t (Founded)  (cost=4.00 rows=1) (actual time=0.187
   rows=15 pages=17)
         Index Cond: Founded = 2005
Rows: 3
Planning Time: 1.131 ms
Execution Time: 2.400 ms
```

El costo se expresa en páginas estimadas. Para un índice no agrupado cuenta la altura del árbol más una página de datos por fila, que es justamente lo que lo vuelve caro en rangos anchos.

4.8. Transacciones y concurrencia

El motor reconoce `BEGIN TRANSACTION`, `END TRANSACTION` (o `COMMIT`) y `ROLLBACK`. El control de concurrencia usa locks a nivel de tabla: un `SELECT` pide un lock compartido (S) y un `INSERT` o `DELETE` pide uno exclusivo (X). Se aplica 2PL estricto, es decir, los locks de una transacción se

conservan hasta que termina, y la espera se implementa con variables de condición, sin espera activa.

Para los deadlocks se optó por detección. Cuando una transacción debe esperar, el gestor arma el grafo de espera y busca un ciclo con una búsqueda en profundidad. Si lo encuentra, aborta a la transacción más joven del ciclo y registra el ciclo y el motivo. Como las páginas se escriben en disco en el momento, el rollback es un deshacer lógico: cada `INSERT` y `DELETE` de la transacción queda anotado y `ROLLBACK` aplica la operación inversa en orden contrario.

La demostración con hilos se describe en la sección 8.5.

4.9. Interfaz de usuario

La interfaz web tiene los cuatro paneles pedidos: archivos (tablas, esquema, índices y estadísticas de páginas), consultas (editor con resaltado de SQL), resultados (tabla con contador y tiempo) y plan de ejecución, que dibuja el árbol de operadores como grafo con las filas, el tiempo y el índice usado en cada nodo. Además incluye la carga de CSV con elección de clave primaria e índices, y una sección de comparación que ejecuta la misma consulta sobre las distintas organizaciones y grafica el resultado.

5. Parte 2: base de datos espacial

5.1. Métricas de distancia

El módulo `motor/espacial/distancia.h` implementa las dos métricas pedidas y devuelve metros:

- Haversine (geodésica), la distancia sobre una esfera de 6 371 km de radio. Es la que se usa por defecto.
- Euclidiana, la distancia en el plano de grados, convertida a metros con el mismo radio.

Caso	Haversine	Euclidiana
1° de latitud, o 1° de longitud sobre el ecuador	111 194.9 m	111 194.9 m
Lima centro a San Isidro	5 745.3 m	5 747.2 m
Lima a Cusco	574.6 km	588.0 km
1° de longitud a latitud 60°	55 596.9 m	111 194.9 m

Tabla 1: Distancias verificadas por `distancia_test`.

La última fila muestra el límite de la métrica euclidiana: trata un grado de longitud como si midiera lo mismo en cualquier latitud, cuando a 60° mide la mitad. Cerca del ecuador, como en Lima, las dos casi coinciden.

5.2. Extensión SQL

El parser se extendió con el tipo `POINT`, el literal `POINT(lat, lon)` y la función `distancia`:

```
CREATE TABLE tiendas (id INT PRIMARY KEY, nombre VARCHAR(20), ubicacion POINT);
INSERT INTO tiendas VALUES (1, 'Centro', POINT(-12.0464, -77.0428));
```

```
SELECT * FROM tiendas WHERE distancia(ubicacion, POINT(-12.0464, -77.0428)) < 5000;
SELECT * FROM tiendas ORDER BY distancia(ubicacion, POINT(-12.0464, -77.0428)) LIMIT
    10;
```

Sin índice, la consulta por radio se resuelve como un filtro sobre la distancia y la de k vecinos ordena por distancia con el mismo merge sort externo del `ORDER BY`, cortando con `LIMIT`. Con un índice R-Tree, ambas se resuelven con el índice, como se explica en la siguiente subsección. Antes de ejecutar se valida que `distancia` reciba una columna `POINT` y que una columna `POINT` no se compare con `=` o `<`. Con cinco puntos de Lima, la consulta de los tres más cercanos al centro devuelve centro, San Isidro y Callao, y la de radio de 6 km devuelve centro y San Isidro, en las tres organizaciones de archivo.

5.3. Índice R-Tree

El R-Tree guarda los puntos en nodos de 4 KB, igual que los demás índices, con 170 entradas por nodo. Cada entrada tiene un rectángulo mínimo envolvente (MBR) en microgrados y una referencia: en una hoja es la posición del registro y en un nodo interno es la página del hijo. Para insertar se baja por la entrada cuyo MBR crece menos, y cuando un nodo se llena se divide con el split cuadrático de Guttman, que elige como semillas las dos entradas que más área desperdiciarían juntas y reparte el resto según cuánto crece cada grupo.

Las consultas usan las métricas del módulo de distancias:

- Rango por radio: se calcula la caja de grados que contiene al círculo y solo se baja por los nodos cuyo MBR la corta; en las hojas se filtra con la distancia exacta.
- k vecinos: búsqueda *best-first* con una cola de prioridad ordenada por la distancia mínima del punto a cada MBR. Como esa distancia nunca supera la de un punto dentro del rectángulo, los puntos salen de la cola ya ordenados del más cercano al más lejano.

Desde SQL, `CREATE INDEX idx_geo ON tiendas (ubicacion) USING RTREE` construye el índice sobre una columna `POINT` de una tabla `HEAP` y el motor lo mantiene en cada `INSERT` y `DELETE`. El planificador lo usa para `distancia(...) <r` y para `ORDER BY distancia(...) LIMIT k`, y el plan lo muestra así:

```
-> Index Scan using idx_geo on tiendas (ubicacion)  (cost=4.00 rows=1) (actual time
    =0.041 rows=2 pages=3)
   Index Cond: distancia(ubicacion, POINT(-12.046400, -77.042800)) [haversine] < 6000
```

5.4. Intersección con polígono

La tercera consulta del enunciado, puntos dentro de un polígono, se escribe así:

```
SELECT nombre FROM tiendas
WHERE dentro(ubicacion, POLYGON((-12.13 -77.05, -12.13 -77.02, -12.09 -77.02, -12.09
    -77.05)));
```

La pertenencia se decide con *ray casting*: desde el punto se traza un rayo y se cuentan los lados del polígono que cruza; si son impares, el punto está dentro. Las cuentas se hacen en microgrados con enteros de 64 bits, de modo que un punto sobre un borde se detecta de forma exacta y se considera dentro. Si la columna tiene R-Tree, el planificador pide al índice los puntos de la caja que envuelve al polígono y después aplica la prueba exacta solo a esos candidatos. En la prueba, un rectángulo alrededor de San Isidro y Miraflores devuelve esas dos tiendas, y un triángulo con la misma base devuelve solo Miraflores, con y sin índice.

5.5. Estado de la Parte 2

Están terminados el tipo `POINT`, las dos métricas, la extensión SQL, el índice R-Tree conectado al planificador, las consultas por radio, k vecinos y polígono, y la comparación experimental con GiST. La interfaz incluye además un panel de mapa con Leaflet que dibuja los puntos de la tabla sobre OpenStreetMap, resalta las filas que devolvió la consulta y traza el círculo de búsqueda o el polígono usado en el `WHERE`. Los límites se detallan en la sección 9.

6. Parte 4: búsqueda por similitud de imágenes

6.1. Índice IVF

El índice IVF (*Inverted File Index*) busca los k vectores más parecidos sin compararlos todos. Aplica a los vectores la misma idea del índice invertido de texto. En texto, cada término apunta a la lista de documentos que lo contienen y una consulta solo recorre las listas de sus términos. En el IVF, el espacio se divide en `nlist` regiones (celdas de Voronoi), cada una con su centroide y con la lista de los vectores que caen en ella. Una consulta solo recorre las listas de las `nprobe` regiones más cercanas. Sivic y Zisserman aplicaron el archivo invertido a imágenes, y Jégou, Douze y Schmid lo usaron como cuantizador grueso delante de una búsqueda exacta.

El profesor aprobó usar el `IndexIVFFlat` de FAISS en lugar de implementarlo. El adaptador `api/multimedia/indices.py` lo envuelve con la misma interfaz que el HNSW (`construir`, `knn`, `insertar`, `eliminar`, `guardar` y `cargar`) y usa como id la clave primaria de la tabla, no la posición del vector.

Construcción. Se entrena un k-means de Lloyd con `nlist` centroides durante 10 iteraciones, con semilla fija para que el índice salga igual en cada corrida. Los centroides se guardan en un índice plano que decide a qué lista va cada vector, y luego cada vector se agrega a la lista de su centroide más cercano. Insertar después es ese mismo último paso, sin reentrenar. Si se insertan muchos vectores de una zona nueva, las listas se desbalancean y conviene reconstruir.

Búsqueda. La consulta se compara con los `nlist` centroides, se eligen los `nprobe` más cercanos y se calcula la distancia exacta solo contra los vectores de esas listas, con un heap de tamaño k. Cada consulta compara unos $\texttt{nlist} + \texttt{nprobe} \cdot n/\texttt{nlist}$ vectores en lugar de n. El precio es el recall: si el vecino verdadero cae en una celda que no se revisó, por ejemplo porque la consulta está cerca del borde entre dos celdas, el IVF no lo encuentra. Con $\texttt{nprobe} = \texttt{nlist}$ se revisan todas las listas y el resultado es el mismo que el de la fuerza bruta, y así lo comprueban las pruebas del adaptador.

Métricas. Las tres métricas del motor se mantienen con la convención de que menor distancia significa más parecido. FAISS devuelve otras escalas y el adaptador las convierte:

Métrica	Índice de FAISS	FAISS devuelve	El adaptador devuelve
Euclidiana	`METRIC_L2`	$\|a-b\|^2$	$\|a-b\|$
Producto punto	`METRIC_INNER_PRODUCT`	$a \cdot b$	$-a \cdot b$
Coseno	`METRIC_INNER_PRODUCT`, normalizados	$\cos(a,b)$	$1-\cos(a,b)$

Tabla 2: Conversión de las distancias de FAISS.

Parámetros y espacio. Se usa `nlist` $\approx \sqrt{n}$ (25, 100 y 316 listas para 1k, 10k y 100k vectores) y `nprobe` se elige midiendo el recall que se necesita. Desde SQL se crea con `CREATE INDEX idx ON productos (v) USING IVF WITH (nlist = 316, nprobe = 8)` y se usa con `SIMILAR_TO(...) USING IVF`. El índice guarda los vectores completos, sin comprimir, más los ids y los centroides: con 100 000 vectores de 256 dimensiones ocupa 98,7 MB, apenas un 1 % más que los vectores solos. El IVF no ahorra espacio, ahorra comparaciones.

7. Interfaz para las Partes 3 y 4

El panel de resultados se adaptó para que cada tipo de consulta se muestre de la forma más fácil de leer. La interfaz reconoce el tipo de consulta a partir del SQL ejecutado y del plan que devuelve el motor, sin cambiar el formato de la respuesta.

- *Texto.* En una consulta con `MATCH(texto, '...')`, la columna del score se muestra con una barra proporcional al mejor resultado, y los términos de la consulta se resaltan en el texto de cada fila. Para resaltar, la interfaz reduce cada palabra a una raíz aproximada (quita los plurales como el paso 1a de Porter y los sufijos *-ing*, *-ed* y *-ly*), de modo que buscar *cars* resalta también *car*. Las palabras vacías (*stopwords*) no se resaltan.
- *Imágenes.* En una consulta por similitud (`SIMILAR_TO`, o un k-NN de vectores con `ORDER BY distancia(...)` y `LIMIT`), los resultados se muestran como una galería de miniaturas ordenadas por similitud, con el nombre del producto, su score y su posición en el ranking. Un botón permite volver a la tabla. Las miniaturas las sirve la API en `/api/imagenes/{id}`, que busca la ruta de la foto del producto en el CSV y la lee de la carpeta local de imágenes.
- *k vecinos espaciales.* En el mapa, una consulta `ORDER BY distancia(lugar, POINT(...)) LIMIT k` dibuja el punto de consulta, numera los vecinos según su posición en el ranking y traza una línea desde el centro hasta cada uno.
- *Plan de ejecución.* El grafo del plan muestra con nombres legibles los nodos nuevos de las Partes 3 y 4 (búsqueda de texto, k-NN de vectores y vecinos con índice de FAISS), junto con el índice y la métrica que usó cada uno.

Las pruebas de Playwright de `web/tests/resultados.spec.ts` cubren el score y el resaltado con una respuesta simulada y con un `MATCH` real sobre `productos_1k`, la galería de un k-NN de vectores y los ejemplos de texto, espacio y vectores del editor.

8. Comparación experimental

8.1. Metodología

Todas las mediciones se hicieron en la misma máquina, con el dataset `organizations` de 1 000, 10 000 y 100 000 filas y la semilla 42. Se reportan dos medidas: el tiempo de reloj y las páginas leídas o escritas, contadas por el propio motor. Los tiempos incluyen el caché de archivos del sistema operativo, así que reflejan CPU y llamadas al sistema más que la latencia de un disco real. Por eso las páginas son la medida más estable para comparar estructuras.

8.2. Heap File frente a archivo secuencial

	Heap File			Secuencial		
Operación	1k	10k	100k	1k	10k	100k
Carga masiva (ms)	1.7	16.1	111.4	8.0	15.0	117.9
Inserción una a una (ms)	20.7	167.0	1 480.2	126.6	824.4	8 330.9
Búsqueda por clave (µs)	89	877	11 267	35	100	213
Páginas por búsqueda	16	158	1 598	6	20	35
Recorrido completo (ms)	0.3	15.3	48.2	1.2	8.7	100.6
Reorganización (ms)	0.2	1.3	16.0	14.1	15.9	84.3
Espacio en disco (MB)	0.15	1.47	14.6	0.18	1.75	18.1

Tabla 3: Heap File frente a archivo secuencial (medido el 4 de octubre de 2026).

El heap inserta con una lectura y una escritura de página, y por eso es entre 5 y 6 veces más rápido al insertar fila por fila. El secuencial paga la búsqueda binaria de cada inserción, el área auxiliar y 20 reorganizaciones automáticas con 100 000 filas. A cambio, el secuencial responde una búsqueda por clave leyendo 35 páginas en lugar de 1 598, unas 50 veces menos.

Esta comparación explica la observación del primer avance sobre el tiempo del heap. El crecimiento no se debe a un error: sin índice, buscar por clave primaria en un heap es un recorrido, y su costo crece de forma lineal con el tamaño del archivo. El secuencial ocupa un 24 % más de disco porque sus páginas quedan al 80 % tras reorganizar. Con carga masiva, ambos tardan lo mismo, porque cada página se escribe una sola vez.

Una segunda medición, tomada como la mediana de 5 corridas (`datos/resultados/heap_vs_secuencial.md`), confirma el mismo comportamiento: la búsqueda por clave en el heap pasa de 16 a 160 y a 1 599 páginas al multiplicar los datos por 10, mientras que el secuencial pasa de 6 a 20 y a 35 páginas. Esa medición también muestra que el orden de llegada importa en el secuencial: con claves aleatorias el área auxiliar crece más (25 629 registros con 100 000 filas frente a 10 625 con claves ascendentes) y la búsqueda lee 115 páginas en lugar de 35.

8.3. Índices: B+ agrupado, B+ no agrupado y hash extensible

Las seis gráficas siguientes comparan los tres índices sobre la misma clave y con registros de 180 bytes. Las mediciones del hash corresponden a la versión en memoria, que fue la usada en el primer avance. La comparación se repitió después con el hash ya en disco (`datos/resultados/bplus_agrupado_vs_no_agrupado_vs_hash.md`, mediana de 5 corridas): contando la carga del heap, los tres índices se construyen en un tiempo parecido, entre 780 y 850 ms con 100 000 filas, y la búsqueda por igualdad tarda 8,0 µs con el hash, 9,2 µs con el B+ agrupado y 11,0 µs con el B+ no agrupado. El hash en disco lee siempre 2 páginas de índice (directorio y bucket) y 1 del heap.

- *Igualdad.* Los tres responden entre 6 y 12 µs. Es la única operación donde el hash gana, y lo hace por poco.
- *Rango.* Para 100 claves, el agrupado lee 12 páginas (hojas consecutivas) y el no agrupado 104, porque paga un salto al heap por cada fila. El hash no soporta rangos y debe recorrer todo el heap: 4 546 páginas con 100 000 filas.
- *Ordenamiento.* El agrupado devuelve las 100 000 filas ordenadas en 18 ms recorriendo sus hojas; el no agrupado tarda 136 ms y el hash 153 ms, porque además debe ordenar.

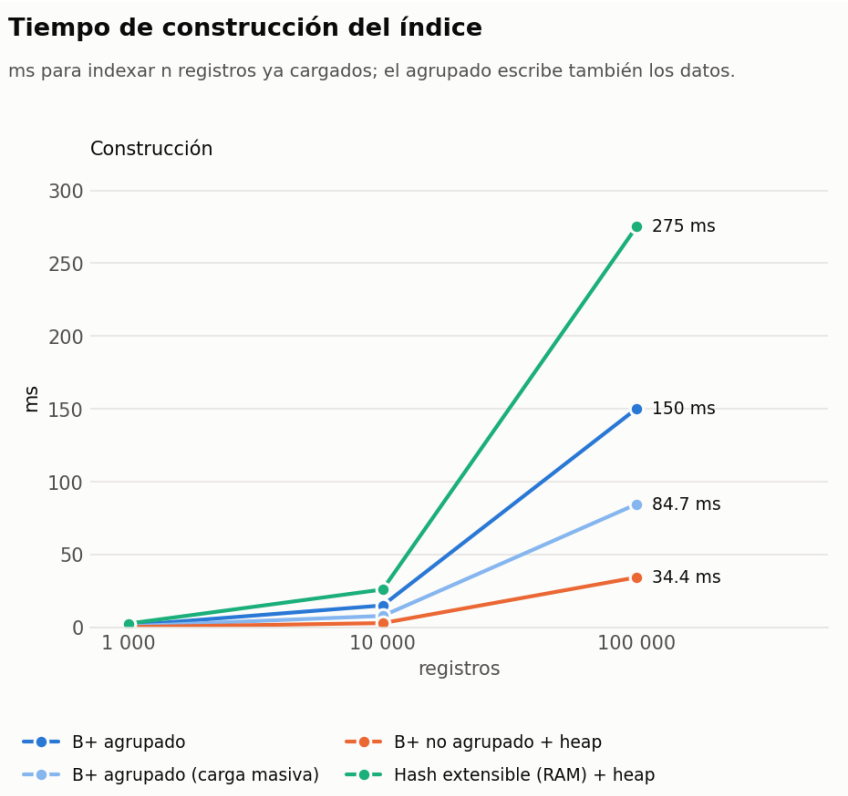


Figura 2: Tiempo de construcción del índice.

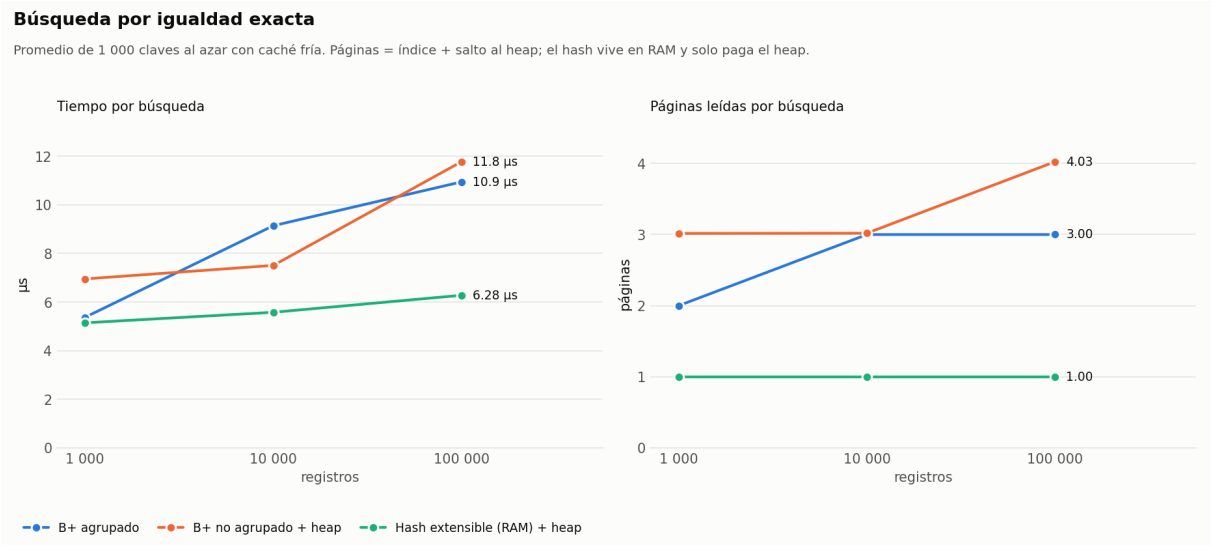


Figura 3: Búsqueda por igualdad: tiempo y páginas leídas.

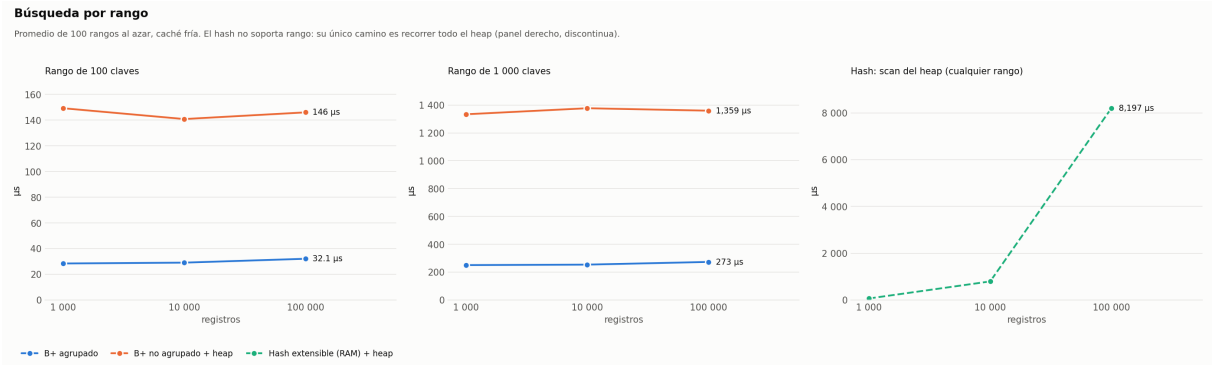


Figura 4: Búsqueda por rango de 100 y 1 000 claves.

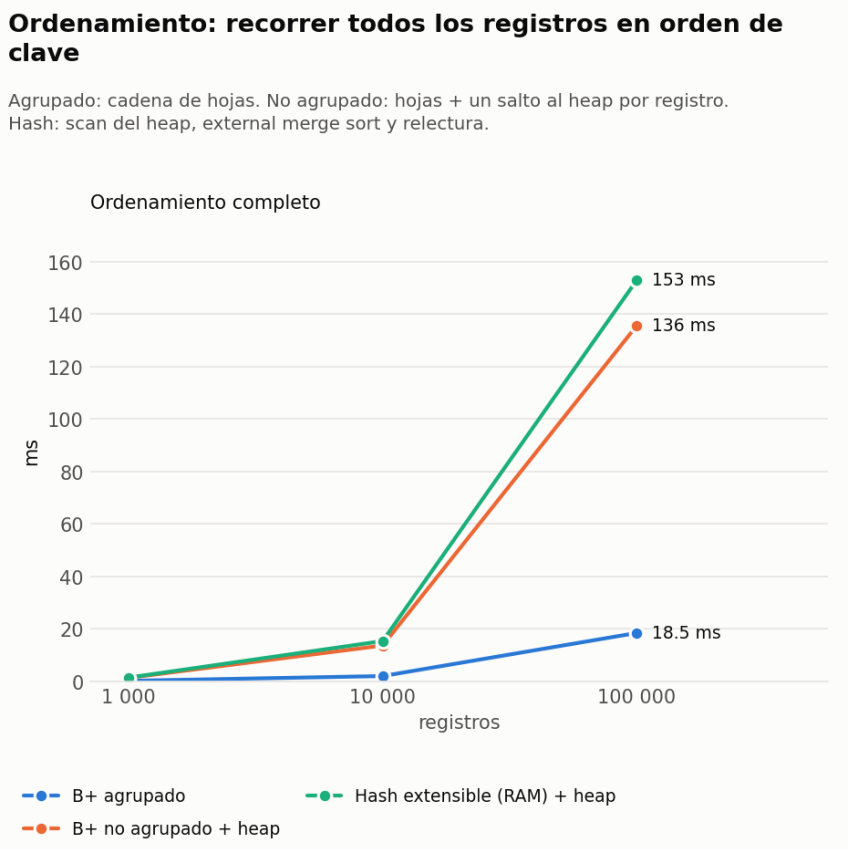


Figura 5: Ordenamiento completo por clave.

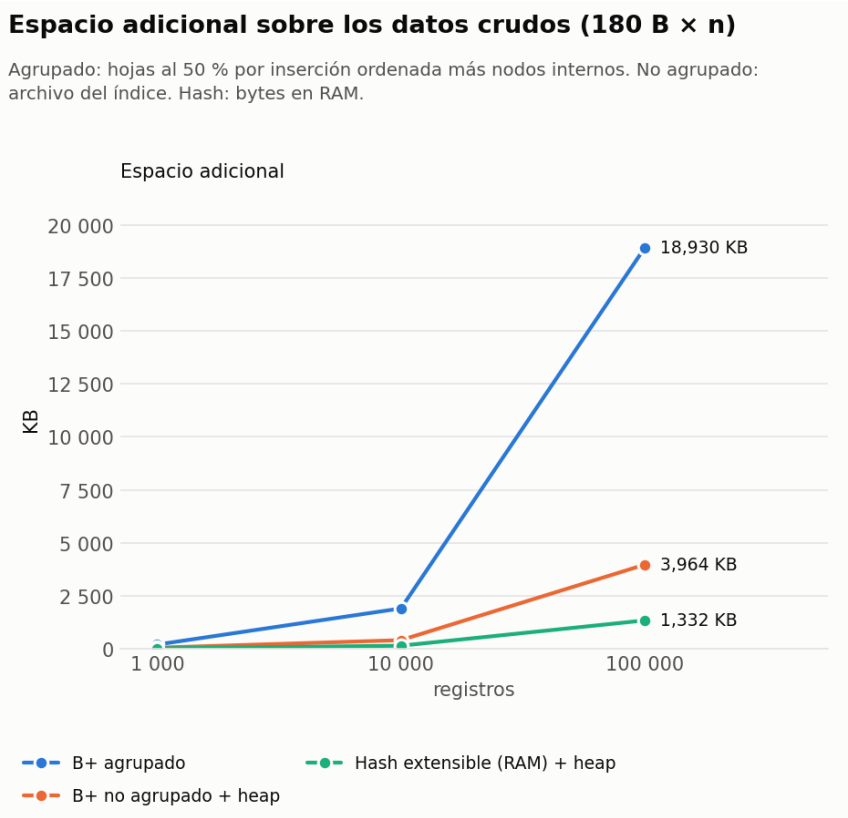


Figura 6: Espacio adicional sobre los datos.

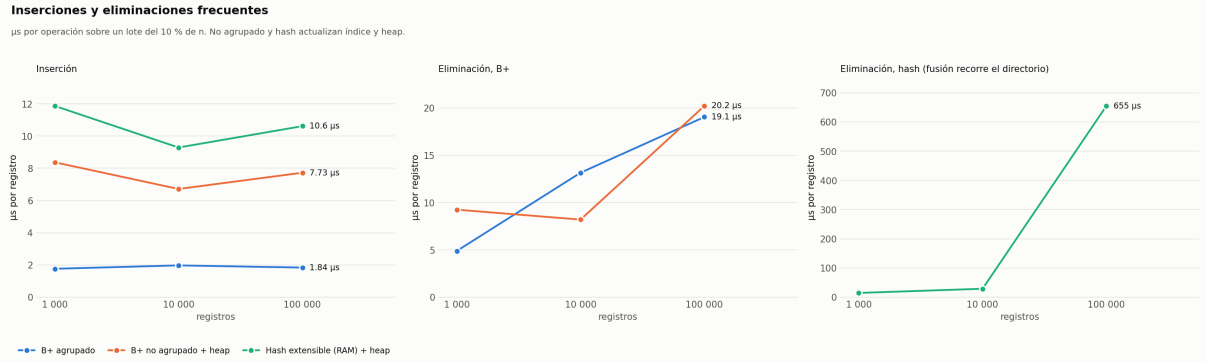


Figura 7: Inserciones y eliminaciones frecuentes.

- *Construcción.* En la gráfica el agrupado parece más lento (150 ms frente a 34 ms del no agrupado con 100 000 filas), y esa fue una observación del primer avance. La diferencia está en lo que se mide: el agrupado escribe las filas completas, mientras que el tiempo del no agrupado no incluye cargar el heap (680 ms adicionales). Sumando datos e índice, el agrupado es unas cinco veces más rápido, y con carga masiva baja a 85 ms.
- *Inserción.* El agrupado inserta en 1.8 µs porque mantiene una sola estructura; los otros dos escriben en el heap y en el índice, entre 8 y 11 µs.

8.4. B+ agrupado frente a B+ no agrupado a través de SQL

La misma comparación se repitió con el motor SQL completo, creando una tabla `USING BPLUS` y otra `USING HEAP` con índices secundarios, sobre 100 000 filas:

Operación	B+ agrupado	Heap + B+ no agrupado
Construcción	349 ms	396 ms
Búsqueda puntual	0.26 ms, 5 páginas	5.6 ms, 4 páginas
Rango del 3 %	9.0 ms, 237 páginas	16.7 ms, 3 033 páginas
Recorrido completo	166 ms, 7 698 páginas	157 ms, 3 709 páginas
Tamaño en disco	31.6 MB	15.2 MB

Tabla 4: B+ agrupado frente a heap con B+ no agrupado, 100 000 filas.

El agrupado gana en búsquedas y rangos porque la fila está en la hoja. Pierde un poco en los recorridos completos porque sus hojas guardan registros de tamaño fijo y ocupa el doble de páginas.

8.5. Transacciones

La demostración con hilos (`make demo-transacciones`) lanza 4 hilos que hacen 5 depósitos de 100 cada uno sobre la misma cuenta. Cada depósito lee el saldo, espera 5 ms y escribe el saldo nuevo. Se corre dos veces:

```
sin transacciones:   esperado 2000, obtenido 500, depositos perdidos 15
con BEGIN/END y 2PL: esperado 2000, obtenido 2000, deadlocks detectados 31
```

Sin transacciones, cada sentencia es atómica, pero entre la lectura y la escritura otro hilo lee el mismo saldo y uno de los depósitos se pierde (*lost update*). En las corridas realizadas se perdieron entre 13 y 15 de los 20 depósitos. Con `BEGIN ... END`, el lock compartido de la lectura se mantiene hasta escribir; cuando dos hilos leen y luego quieren escribir, se produce un deadlock que el gestor detecta, la víctima se deshace y reintenta, y el saldo final siempre es 2 000. El número exacto de deadlocks varía entre corridas porque depende de cómo el sistema operativo reparte los hilos, pero la conclusión no cambia. Los eventos de locks se exportan a `datos/resultados/transacciones_eventos.json` para que la interfaz pueda dibujarlos.

8.6. Parte 2: R-Tree frente a búsqueda secuencial

La prueba `rtree_test` inserta 100 000 puntos al azar alrededor de Lima (semilla 42) y compara el R-Tree con una búsqueda secuencial sobre los mismos puntos:

Medida	Valor
Altura del árbol	3
Nodos	872
Tamaño en disco	3 492 KB
Nodos visitados, radio de 1 km	14 de 872
Nodos visitados, k = 10	8 de 872

Tabla 5: R-Tree con 100 000 puntos.

Las 60 consultas de rango (radios de 1, 5 y 10 km con las dos métricas) y las 30 de k vecinos (k = 10, 50 y 100) devuelven exactamente lo mismo que la búsqueda secuencial. La diferencia está en lo que se lee: la búsqueda secuencial siempre recorre los 100 000 puntos, mientras que el R-Tree resuelve un radio de 1 km leyendo 14 nodos y los 10 vecinos más cercanos leyendo 8.

8.7. Parte 2: secuencial, R-Tree y GiST

Para la sección 2.2.4 se midieron tres técnicas sobre los mismos puntos: la búsqueda secuencial y el R-Tree a través del motor SQL (`make bench-espacial`), y el índice GiST de PostgreSQL 18 sobre el tipo `point` nativo (`datos/resultados/gist_bench.sql`). Los puntos se generan con el mismo generador congruencial en C++ y en SQL, y las sumas de coordenadas coinciden en los dos lados. Cada tiempo es el promedio de 100 consultas con la métrica euclidiana, que es la que calcula GiST sin PostGIS. En los 18 casos, las tres técnicas devuelven el mismo número de filas.

Consulta (100 000 puntos)	Filas	Secuencial	R-Tree	GiST
Rango de 1 km	126	52,8 ms	5,7 ms	0,54 ms
Rango de 5 km	2 895	53,5 ms	25,4 ms	2,82 ms
Rango de 10 km	10 370	54,2 ms	73,7 ms	7,16 ms
k-NN, k = 10	10	342,6 ms	5,0 ms	0,19 ms
k-NN, k = 50	50	326,4 ms	5,1 ms	0,62 ms
k-NN, k = 100	100	409,5 ms	5,7 ms	0,56 ms
Construcción del índice			5 116 ms	424 ms
Tamaño del índice			3 448 KB	6 528 KB

Tabla 6: Búsqueda secuencial, R-Tree del motor y GiST de PostgreSQL con 100 000 puntos.

La búsqueda secuencial cuesta lo mismo con cualquier radio, porque siempre lee las 491 páginas del heap. El R-Tree es unas 9 veces más rápido con un radio de 1 km, pero su costo crece con el número de filas encontradas, ya que por cada una lee una página del heap. Con 10 km la consulta devuelve el 10 % de la tabla y el R-Tree termina siendo más lento que el recorrido, el mismo efecto que se observó con el B+ no agrupado en rangos anchos. En k vecinos la diferencia es mucho mayor: la búsqueda secuencial calcula la distancia de todos los puntos y ordena la tabla completa, mientras que el R-Tree solo baja por los nodos más cercanos y es entre 64 y 71 veces más rápido, casi sin depender de k.

GiST resultó unas 10 veces más rápido que el R-Tree del motor en todas las consultas, aunque su índice ocupa casi el doble. Parte de esa diferencia no viene de la estructura: cada consulta del motor abre los archivos y lee el catálogo, mientras que PostgreSQL trabaja con la tabla ya en memoria compartida. Por eso la comparación más justa es la forma en que crece cada técnica, y en eso el R-Tree y GiST se comportan igual. La construcción del R-Tree es más lenta porque inserta punto por punto leyendo cada fila del heap. El detalle completo está en `docs/comparacion_espacial/README.md`.

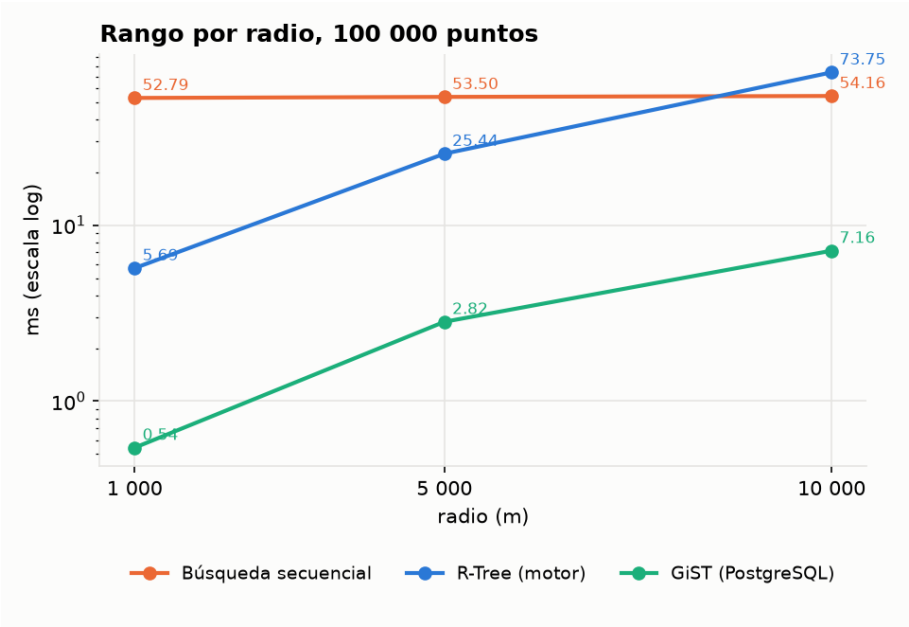


Figura 8: Rango por radio con 100 000 puntos (escala logarítmica).

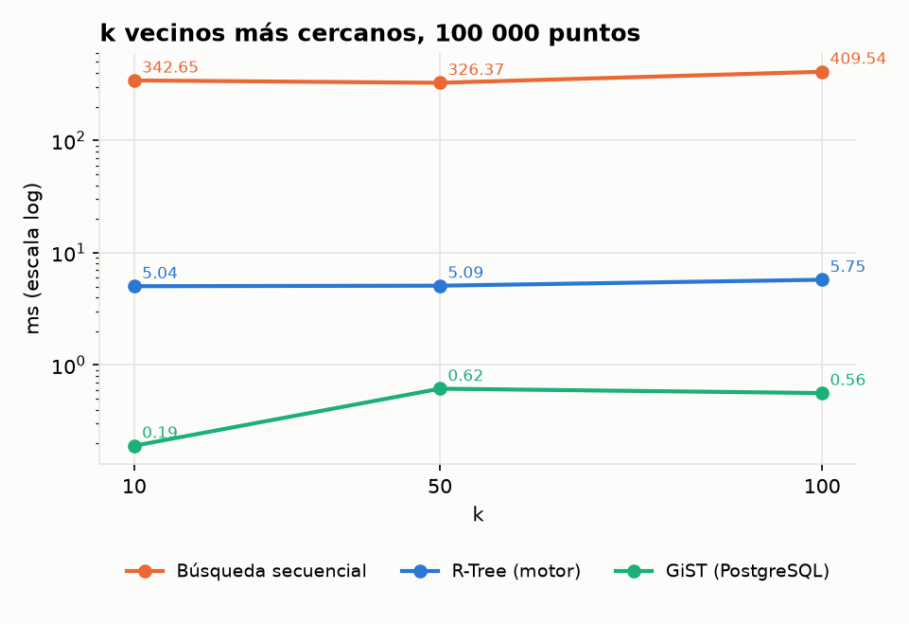


Figura 9: k vecinos más cercanos con 100 000 puntos (escala logarítmica).

8.8. Parte 3: TF-IDF, BM25 y GIN

Se comparó el índice invertido SPIMI del motor, con ranking TF-IDF (coseno lnc.ltc) y BM25 ($k_1 = 1{,}2$, $b = 0{,}75$), contra el índice GIN de PostgreSQL 18 con `ts_rank`, sobre la columna `descripcion` de los productos de ABO.

Metodología. El programa `motor/pruebas/texto_bench.cpp` (`make bench-texto`) construye el índice SPIMI con bloques de 4 MB y mezcla k-way, lo vuelve a abrir desde disco y consulta directamente el índice, sin la capa SQL. En PostgreSQL, `datos/resultados/gin_bench.sql` crea una columna `tsvector` generada con `to_tsvector('english', descripcion)`, el índice GIN, y consulta con `@@` y `ts_rank` dentro de un bloque PL/pgSQL. Se usan 30 consultas fijas, 10 de cada largo (1, 3 y 5 o más palabras), iguales en los dos lados. Cada una se repite 5 veces después de una pasada de calentamiento y se reporta el promedio. TF-IDF y BM25 rankean todo documento que tenga al menos una palabra de la consulta, así que en GIN la consulta de `plainto_tsquery`, que une las palabras con AND, se convierte a OR para responder lo mismo.

	Construcción			Tamaño	
n	SPIMI	GIN	GIN: carga + `tsvector`	SPIMI	GIN
1 000	99 ms	21 ms	68 ms	0,5 MB	0,4 MB
10 000	0,81 s	0,16 s	0,73 s	4,9 MB	2,1 MB
100 000	8,2 s	1,23 s	7,7 s	48,3 MB	15,2 MB

Tabla 7: Construcción y espacio del índice de texto.

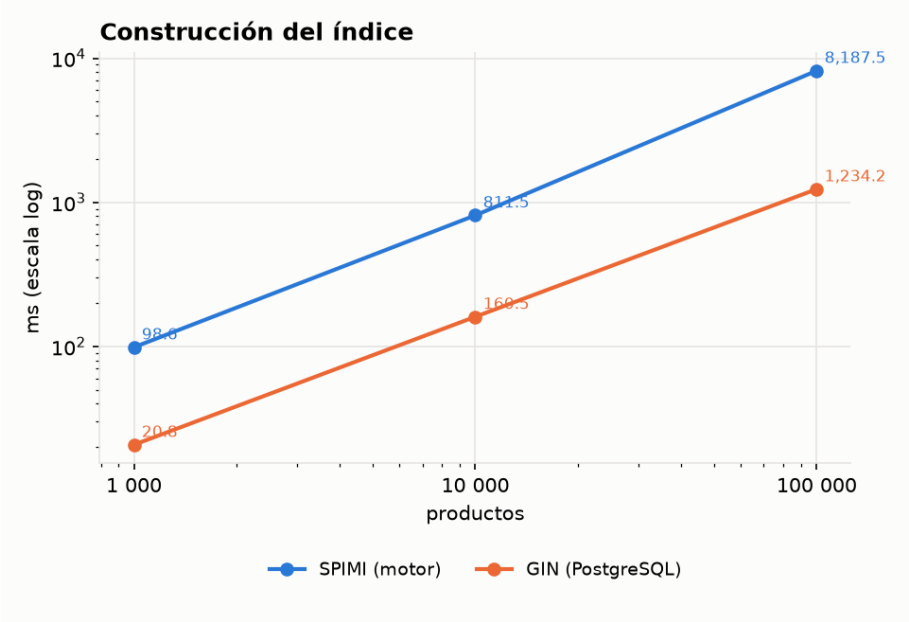


Figura 10: Tiempo de construcción del índice de texto.

Los dos índices crecen de forma lineal con n. GIN construye casi 7 veces más rápido, pero porque el `tsvector` ya está calculado: la tokenización y el stemming se pagan al cargar la tabla, y sumando carga e índice PostgreSQL tarda casi lo mismo que SPIMI, que tokeniza durante la construcción. GIN ocupa un tercio porque comprime sus listas de postings (diferencias entre ids con codificación de largo variable) y no guarda la frecuencia, mientras que SPIMI guarda cada posting sin comprimir, con un `doc_id` de 8 bytes y un `tf` de 4 bytes.

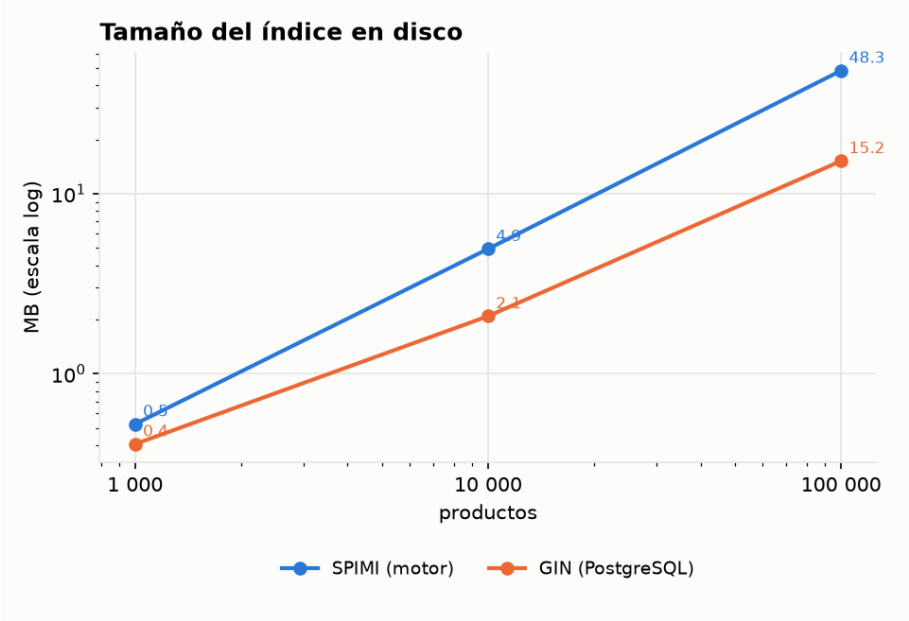


Figura 11: Tamaño del índice de texto.

Consulta (100 000 productos)	TF-IDF	BM25	GIN	Postings leídos por SPIMI
1 palabra	1,5 ms	1,8 ms	2,1 ms	1 139
3 palabras	4,4 ms	4,4 ms	17,3 ms	15 723
5 o más palabras	7,9 ms	7,8 ms	48,8 ms	38 138

Tabla 8: Tiempo de consulta del índice de texto con 100 000 productos.

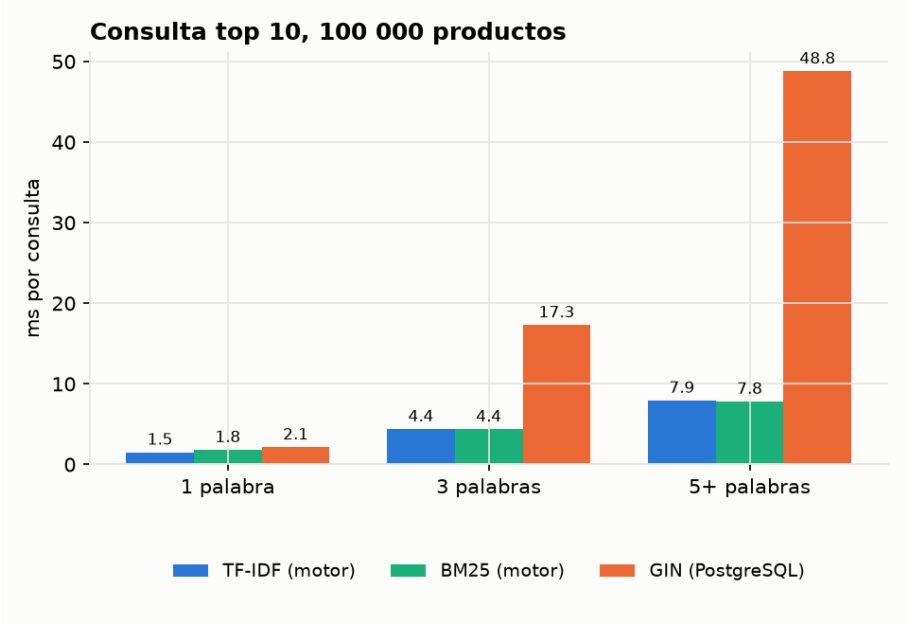


Figura 12: Tiempo de consulta según el número de palabras.

TF-IDF y BM25 tardan lo mismo porque leen exactamente los mismos postings y solo cambia la fórmula del score. El tiempo de SPIMI crece con los postings leídos: con 100 000 productos y 5 palabras recorre 38 mil postings, unas 114 páginas de 4 KB. GIN es parecido con una palabra,

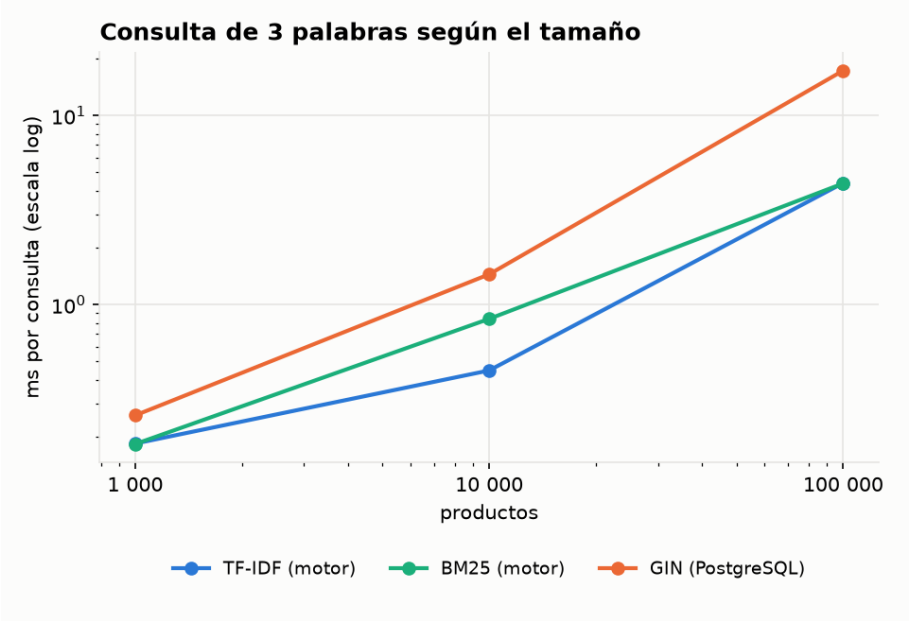


Figura 13: Tiempo de consulta según el tamaño de la colección.

pero entre 4 y 6 veces más lento con varias. El índice GIN solo dice qué filas cumplen `doc @@ q`, y para rankearlas PostgreSQL tiene que leer cada fila candidata y calcular `ts_rank` sobre su `tsvector`; con OR, las candidatas son miles. SPIMI, en cambio, calcula el score desde los postings, que ya traen el `tf`, y se queda con los 10 mejores en un heap sin tocar la tabla.

Calidad del ranking. Para medir la relevancia se escribieron a mano 30 consultas, guardadas en `consultas_relevancia.csv`. Un producto se considera relevante si tiene el mismo `tipo` que la consulta y su `categoria` termina en la misma ruta, por ejemplo *bar stool* con `STOOL_SEATING` en `.../Barstools`. Ninguno de los dos campos está en el texto indexado, así que el juicio no favorece a ninguna técnica. Las tres responden sobre los mismos 10 000 productos.

Técnica	Precision@10	Recall@10
TF-IDF (coseno)	0,360	0,219
BM25	0,507	0,301
GIN (`ts_rank`)	**0,533**	**0,306**

Tabla 9: Calidad del ranking con 30 consultas sobre 10 000 productos. El Recall@10 máximo posible es 0,611, porque varias consultas tienen más de 10 productos relevantes.

BM25 supera a TF-IDF por 15 puntos de precisión. El coseno favorece los documentos muy cortos: para *table lamp*, TF-IDF pone primero "Rivet End Tables", de 16 caracteres, que solo tiene *table*, porque con un vector de norma chica una sola palabra ya da un coseno alto. BM25 satura la frecuencia y normaliza el largo de forma más suave, así que prefiere las lámparas que tienen las dos palabras. GIN y BM25 quedan casi empatados, porque `ts_rank` tampoco penaliza mucho el largo y premia tener varias palabras de la consulta. Los valores absolutos son bajos para las tres técnicas porque el juicio por metadatos es estricto: un producto que responde la consulta pero está en otra categoría cuenta como no relevante. Lo que importa es la comparación, que usa los mismos juicios para todas.

En resumen, BM25 es la mejor opción del motor: tarda lo mismo que TF-IDF y su precisión está

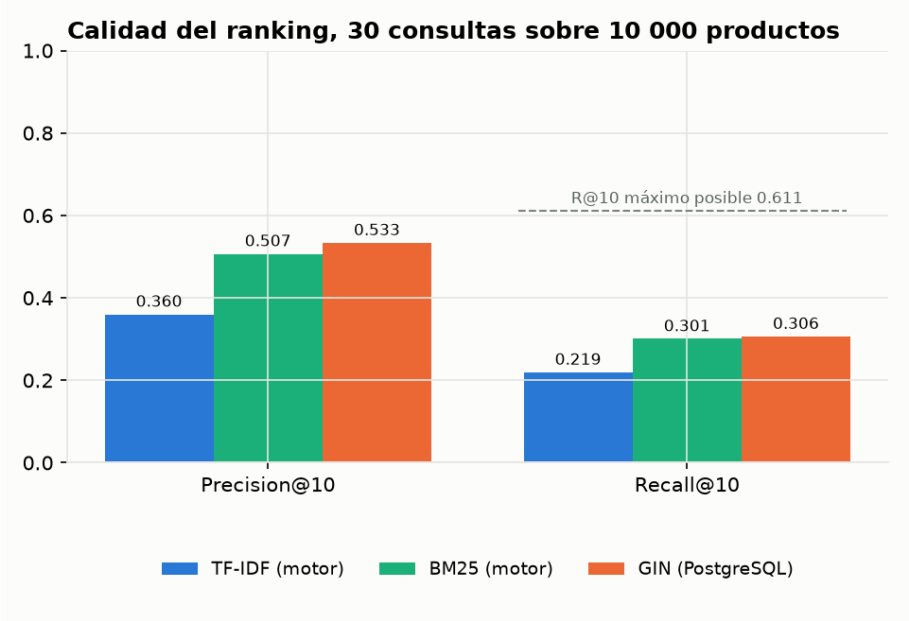


Figura 14: Precision@10 y Recall@10 de las tres técnicas.

al nivel de PostgreSQL. Frente a GIN, SPIMI ocupa unas 3 veces más espacio pero responde las consultas de varias palabras entre 4 y 6 veces más rápido. Como en la Parte 2, los tiempos de los dos sistemas no se miden igual (el motor sin la capa SQL y PostgreSQL con la tabla en memoria compartida), y el stemming no es idéntico (Porter en el motor y Snowball en PostgreSQL), así que la comparación más justa es cómo crece cada uno. El detalle está en `docs/comparacion_texto/README.md`.

8.9. Parte 4: k-NN secuencial, IVF y HNSW

Se comparó el k-NN secuencial del motor (`ORDER BY distancia(v, VECTOR(...)) LIMIT 10`) con los índices IVF y HNSW de FAISS y con el índice plano exacto de FAISS, con las métricas euclidiana y coseno.

Metodología. Los vectores son los histogramas TF-IDF de 256 palabras visuales de las fotos de los productos, con norma 1. Se extraen una vez para los 100 000 productos y se usan los primeros 1 000 y 10 000. Las consultas son 50 vectores de la colección con un ruido gaussiano pequeño ($\sigma = 0{,}01$), para medir una búsqueda de vecinos y no solo que cada vector se encuentre a sí mismo, siempre con $k = 10$. La respuesta correcta la da el índice plano de FAISS, que compara contra todos los vectores. Para el motor, `api/multimedia/bench_vectorial.py -motor` carga los vectores con `COPY` en una tabla `HEAP` y ejecuta cada consulta como un `SELECT`, tomando el tiempo que informa el motor sin contar el arranque del proceso. El IVF usa `nlist` $\approx \sqrt{n}$ y `nprobe` de 1 a `nlist`; el HNSW usa $M = 32$, `ef_construccion` $= 100$ y `ef_busqueda` de 16 a 256. FAISS corre con un solo hilo.

	Coseno		Euclidiana	
Técnica (100 000 vectores)	Tiempo	Recall@10	Tiempo	Recall@10
k-NN secuencial del motor	488 ms	0,996	460 ms	0,994
Plano exacto de FAISS	3,72 ms	1,000	4,41 ms	1,000
IVF, `nprobe` = 8	0,58 ms	0,934	0,60 ms	0,952
IVF, `nprobe` = 32	3,07 ms	0,986	2,03 ms	1,000
HNSW, `ef_busqueda` = 64	0,42 ms	0,978	0,39 ms	0,886
HNSW, `ef_busqueda` = 256	1,16 ms	0,984	1,74 ms	0,944

Tabla 10: Tiempo de consulta y recall con 100 000 vectores.

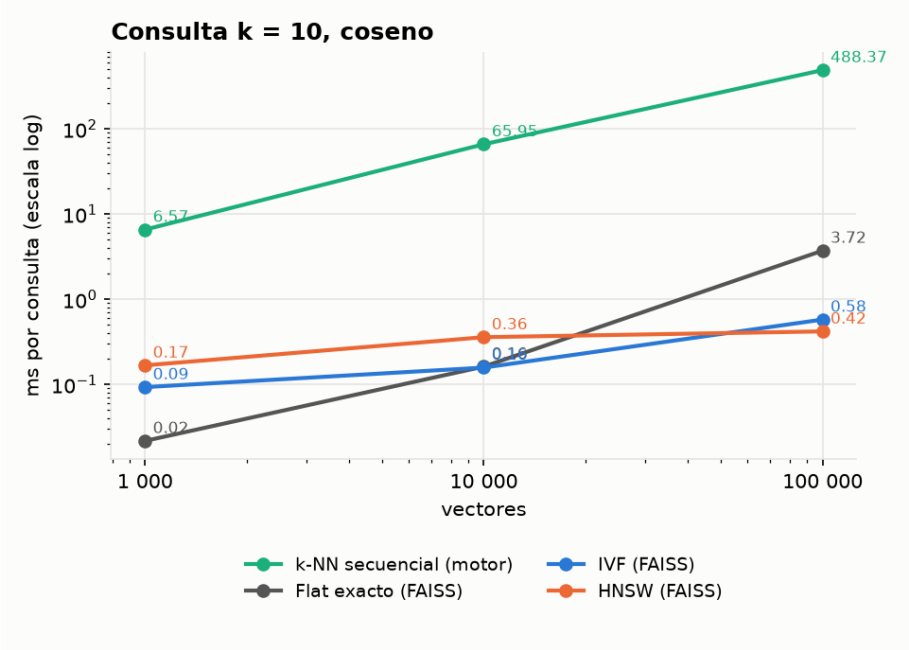


Figura 15: Tiempo de consulta según el tamaño de la colección, métrica coseno.

El k-NN secuencial crece de forma lineal con n (6,6, 66 y 488 ms): lee las 33 334 páginas de la tabla, con 3 vectores de 1 KB por página, y calcula 100 000 distancias. El índice plano de FAISS hace las mismas cuentas, pero en memoria y con instrucciones vectoriales, y es unas 130 veces más rápido; la diferencia es el costo de leer del disco y deserializar cada registro. IVF y HNSW casi no crecen: de 1k a 100k, el IVF pasa de 0,09 a 0,58 ms y el HNSW de 0,17 a 0,42 ms, porque solo revisan una parte de los vectores. Con 100 000 vectores, el HNSW es unas 1 000 veces más rápido que el recorrido del motor. Con 1 000 vectores, en cambio, el plano es el más rápido (0,02 ms), porque con tan pocos datos recorrerlos todos cuesta menos que consultar un índice.

Las curvas de recall contra tiempo muestran cómo cambia cada índice al mover su parámetro. El IVF cambia recall por tiempo de forma predecible: con coseno pasa de 0,73 con `nprobe` = 1 (0,20 ms) a 0,93 con 8 (0,58 ms) y 0,99 con 32 (3 ms); más allá ya cuesta lo mismo que el plano. El HNSW llega antes a un recall alto, 0,92 en 0,22 ms con `ef_busqueda` = 16, pero se estanca: subir a 256 lo deja en 0,98 y multiplica el tiempo por 5. Para el mismo tiempo, el HNSW gana con coseno y el IVF gana con euclidiana, donde llega a 1,0 con `nprobe` = 32 mientras el HNSW se queda entre 0,89 y 0,94.

Euclidiana frente a coseno. Como los histogramas tienen norma 1, se cumple $\|a - b\|^2 = 2 - 2\cos(a, b)$, y las dos métricas ordenan los vecinos igual. Por eso el plano, el motor y el IVF dan

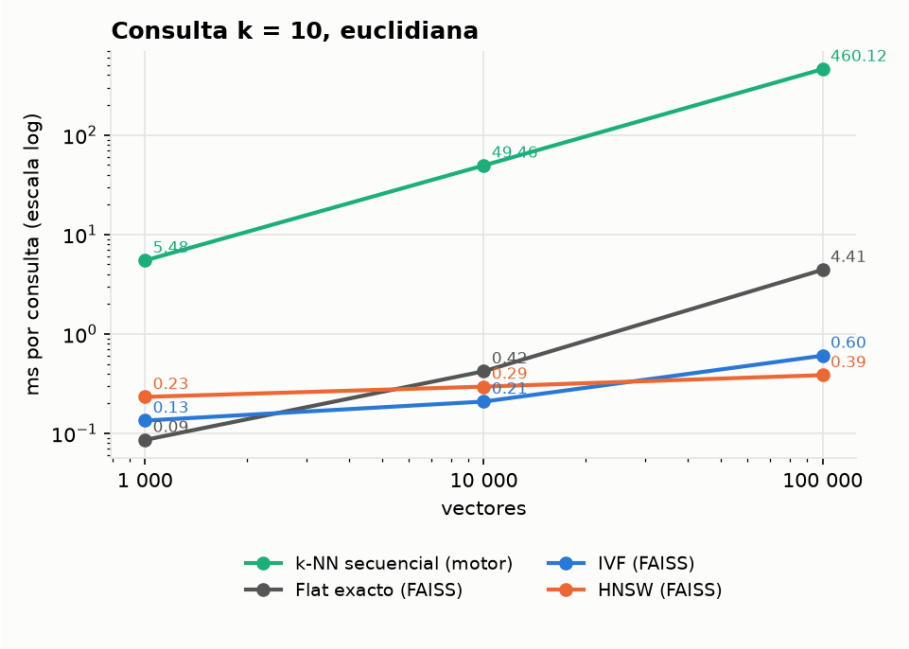


Figura 16: Tiempo de consulta según el tamaño de la colección, métrica euclidiana.

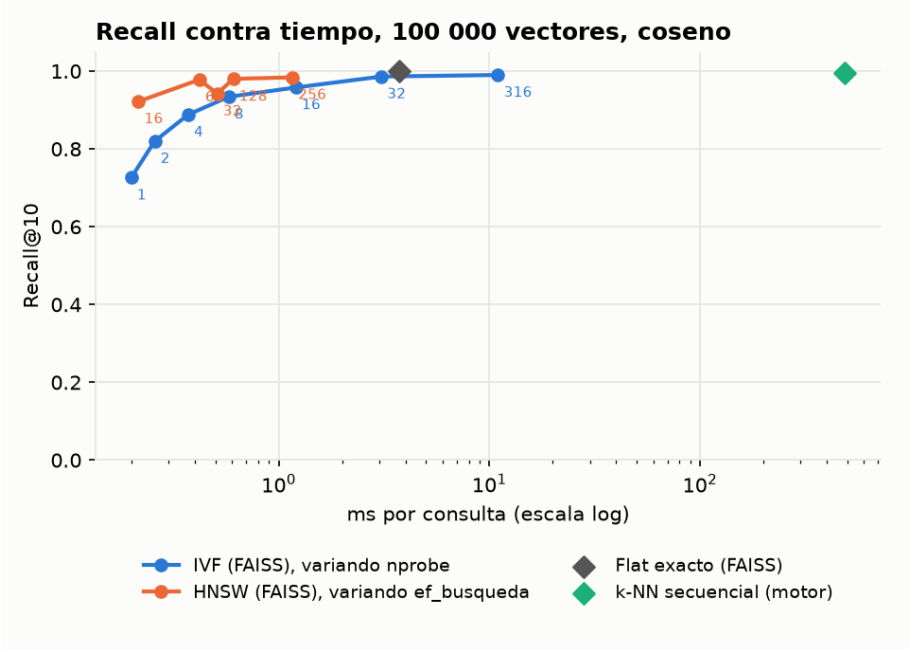


Figura 17: Recall@10 contra tiempo de consulta con 100 000 vectores, métrica coseno.

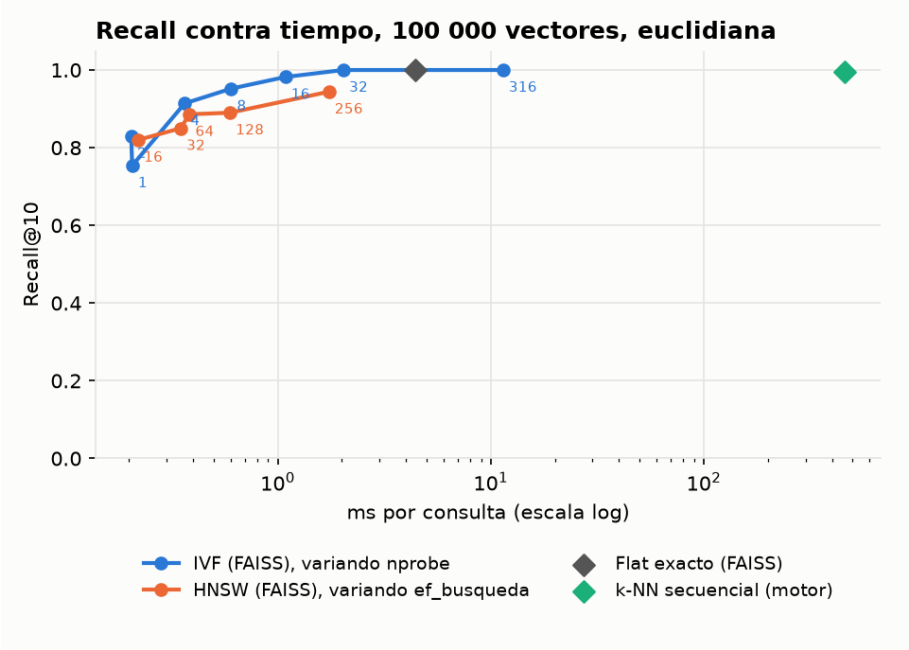


Figura 18: Recall@10 contra tiempo de consulta con 100 000 vectores, métrica euclidiana.

casi el mismo recall con ambas. La diferencia aparece en el HNSW, cuyo grafo se construye con la métrica del índice. Una explicación probable, que no se midió aparte, es que con la euclidiana al cuadrado las distancias entre vectores de norma 1 caen en un rango muy chico, quedan más vecinos casi empatados y la búsqueda voraz se detiene antes. Se recomienda el coseno: compara la proporción de palabras visuales y no su cantidad, rinde mejor con el HNSW y sigue siendo correcto si algún vector llega sin normalizar.

n	IVF: construcción	HNSW: construcción	Plano	IVF	HNSW
1 000	10 ms	98 ms	1,0 MB	1,0 MB	1,2 MB
10 000	0,12 s	0,62 s	9,8 MB	9,9 MB	12,4 MB
100 000	1,8 s	13,5 s	97,7 MB	98,7 MB	124,4 MB

Tabla 11: Construcción y tamaño de los índices vectoriales con la métrica coseno. El k-NN secuencial no construye nada.

El IVF construye unas 7 veces más rápido que el HNSW, porque su costo es entrenar el k-means y asignar cada vector a una lista, mientras que el HNSW inserta los vectores de a uno buscando en el grafo los M vecinos de cada uno. Ninguno de los dos ahorra espacio, porque ambos guardan los vectores completos: el IVF agrega solo los ids y los centroides (1 %) y el HNSW agrega los enlaces del grafo (27 %).

Éxitos y errores. Las figuras siguientes muestran la consulta y sus 5 vecinos más cercanos por coseno sobre los 100 000 productos. El borde verde indica que el vecino es del mismo `tipo` que la consulta y el rojo que es de otro. Se considera éxito cuando al menos 4 de los 5 vecinos son del mismo tipo, y error cuando no lo es ninguno. El dataset guarda solo la foto principal de cada producto, así que el criterio usa el `tipo` y no otras fotos del mismo producto.

Los éxitos son imágenes con mucha textura y un diseño repetido. En el primero, una funda con un guacamayo devuelve el mismo diseño para otros 5 modelos de celular, que es el caso de “otra foto del mismo producto”. Los errores son objetos lisos o blancos, como una funda transparente

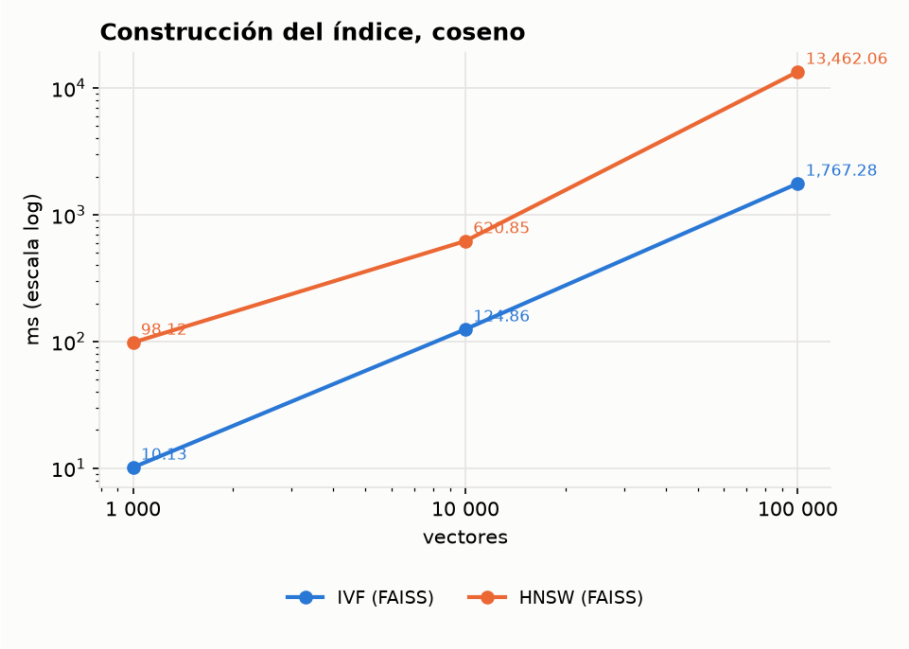


Figura 19: Tiempo de construcción de los índices vectoriales.

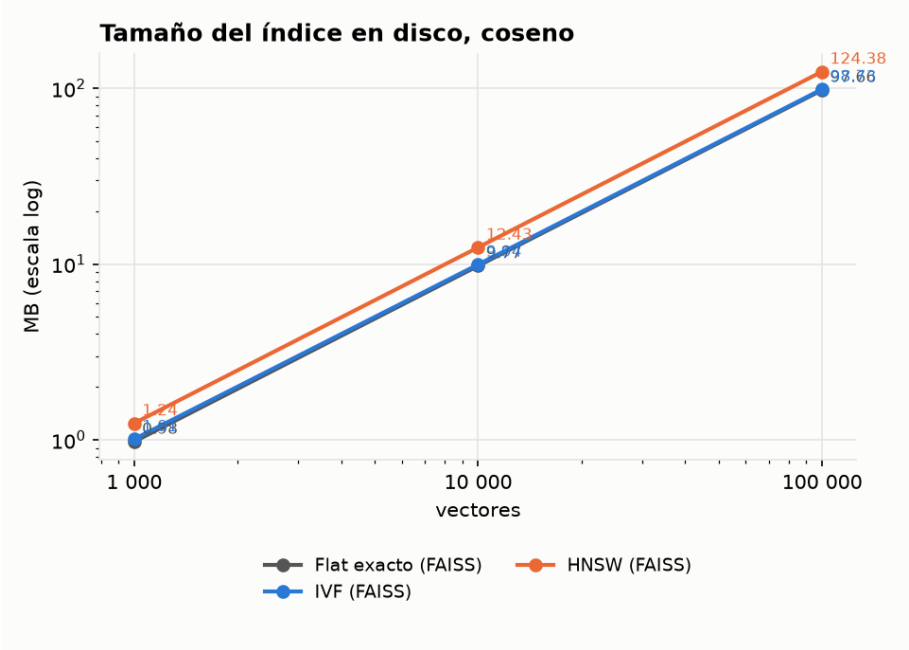


Figura 20: Tamaño de los índices vectoriales.

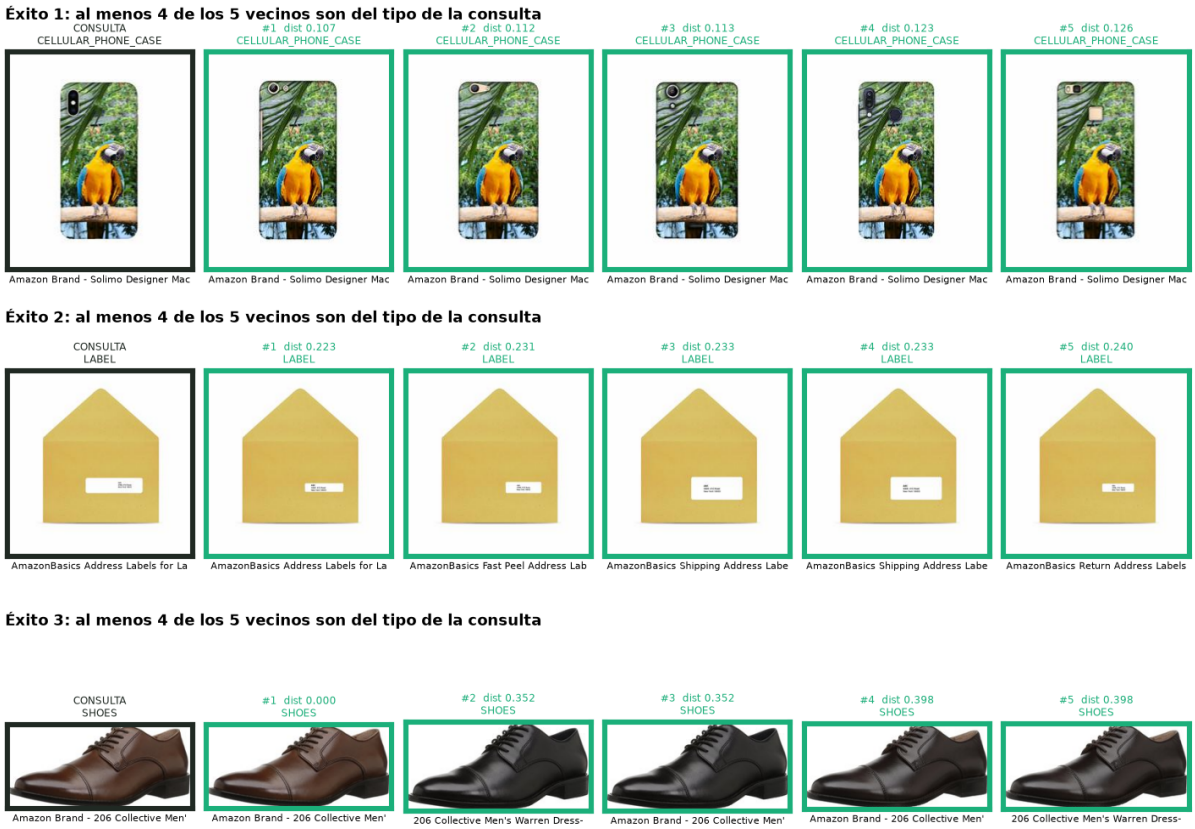


Figura 21: Tres éxitos: la consulta (izquierda) y sus 5 vecinos más cercanos.

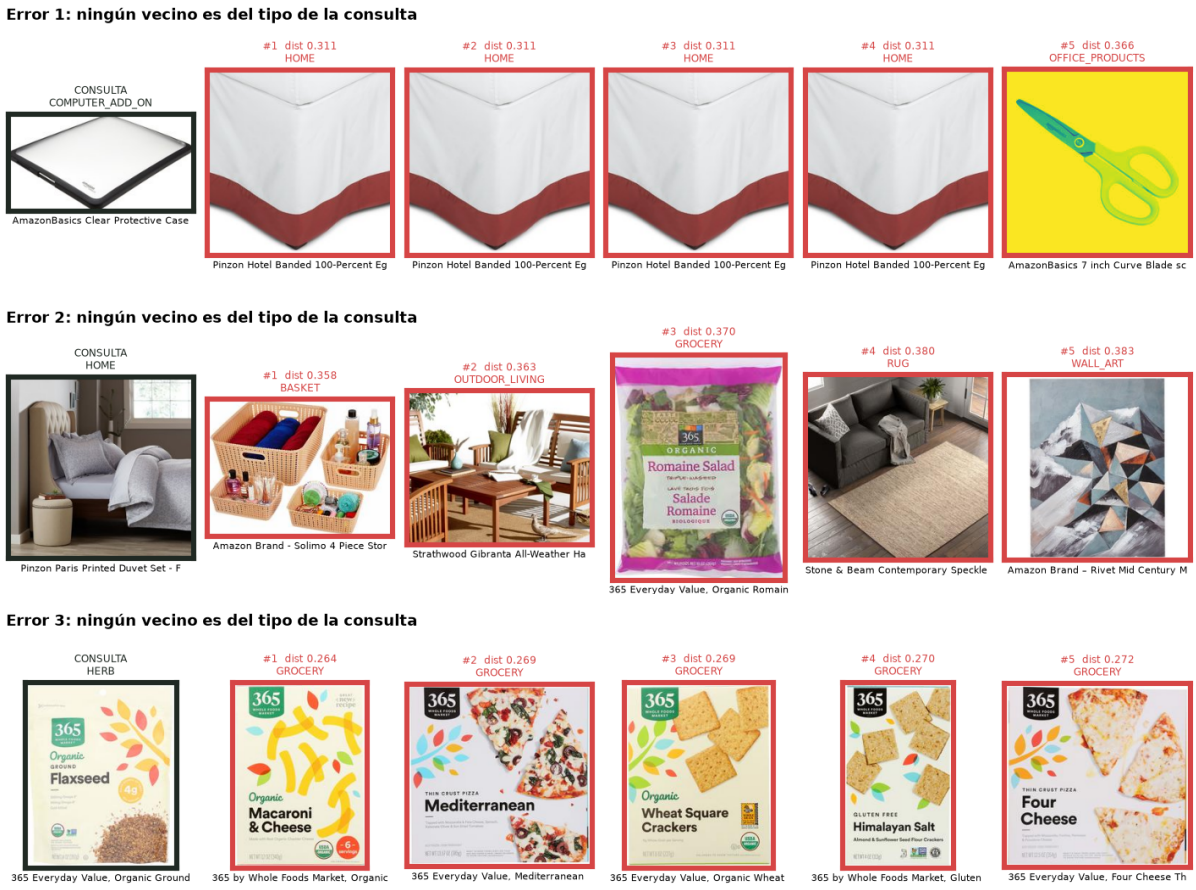


Figura 22: Tres errores: ninguno de los 5 vecinos es del mismo tipo que la consulta.

que devuelve faldones de cama: SIFT encuentra pocos puntos clave en superficies sin textura, y la bolsa de palabras visuales no sabe de forma ni de color, así que las imágenes se parecen por el fondo blanco.

Limitaciones. Los 15 571 productos que comparten foto tienen vectores idénticos. Cuando dos vecinos empatan en el puesto 10, cada técnica desempata distinto, y por eso el motor da un recall de 0,994 a 0,996 y no de 1, aunque su búsqueda es exacta. Además, los tiempos del motor incluyen leer del disco y los de FAISS no, así que la comparación justa entre ambos es cómo crece cada uno con n. El detalle está en `docs/comparacion_vectorial/README.md`.

8.10. Cuándo usar cada técnica

Técnica	Conviene cuando	Desventaja
Heap File	hay muchas inserciones y recorridos completos	buscar por clave sin índice es un recorrido
Secuencial	se busca por clave o por rango y se inserta poco	inserciones lentas y reorganizaciones
B+ agrupado	se consulta por rango o se lista ordenado por la clave primaria	solo uno por tabla; ocupa más páginas
B+ no agrupado	se necesita un índice secundario con repetidos, por igualdad o rango	una lectura del heap por cada fila devuelta
Hash extensible	solo hay búsquedas por igualdad	no sirve para rangos ni para ordenar
R-Tree	se busca por distancia a un punto o dentro de un polígono, con pocas filas en el resultado	radios que devuelven gran parte de la tabla
SPIMI + TF-IDF	los textos tienen un largo parecido	premia los textos muy cortos; el índice ocupa unas 3 veces más que GIN
SPIMI + BM25	se rankea texto de propósito general; es la mejor opción del motor	mismo espacio que TF-IDF; `INSERT` y `DELETE` obligan a reconstruir
GIN (PostgreSQL)	se filtra con AND o la tabla cambia seguido	lento para rankear consultas OR largas
k-NN secuencial	la tabla es chica o la consulta lleva otros filtros en el `WHERE`	lee toda la tabla: unos 0,5 s con 100 000 vectores
IVF	la colección es grande y se reconstruye seguido, o se quiere controlar el recall con `nprobe`	con `nprobe` chico pierde vecinos en los bordes de las celdas
HNSW	búsqueda interactiva con coseno sobre datos que caben en memoria	construcción 7 veces más lenta y 27 % más de memoria

Tabla 12: Resumen de ventajas y desventajas.

9. Conclusiones y límites conocidos

El proyecto permitió comprobar en la práctica varias ideas que en clase se ven de forma teórica. La más clara es que el número de páginas leídas explica casi todas las diferencias de rendimiento: el secuencial gana en búsquedas porque lee 35 páginas en lugar de 1 598, y el B+ agrupado gana en rangos porque sus filas están contiguas. También quedó claro que medir bien es parte del trabajo, porque dos de las observaciones del primer avance se explicaban por qué se estaba midiendo y no por un error en las estructuras.

La Parte 1 está completa, con dos límites. El motor no implementa `JOIN` aunque los algoritmos de hash join e index nested loop join ya están escritos en `external_algorithms.h`; falta conectarlos al parser y al ejecutor. Los locks son a nivel de tabla, lo que simplifica el gestor pero reduce la concurrencia frente a locks por registro.

De la Parte 2 están terminados el tipo `POINT`, las dos métricas de distancia, la extensión SQL, el índice R-Tree con sus tres consultas (radio, k vecinos y polígono), el panel de mapa en la interfaz y la comparación con GiST. La comparación dejó una lección parecida a la de la Parte 1: un índice ayuda mucho cuando la consulta devuelve pocas filas, como en k vecinos o en radios chicos, pero deja de convenir cuando devuelve una parte grande de la tabla, porque cada fila cuesta una página del heap. También mostró que una implementación propia sigue la misma forma que GiST, aunque PostgreSQL es unas 10 veces más rápido por todo lo que optimiza fuera de la estructura. El repositorio ya incluye `postgis_bench.sql`, que repite la comparación con PostGIS sobre `geography` y distancia geodésica; como trabajo futuro queda ejecutarlo y comparar esos tiempos con la métrica haversine del motor.

En las Partes 3 y 4 se repitió la misma lección con otros tipos de datos. En texto, el índice invertido del motor responde las consultas largas más rápido que GIN porque el score sale del propio índice, sin leer la tabla, y BM25 resultó claramente mejor que TF-IDF a igual costo. En vectores, los índices aproximados son entre 100 y 1 000 veces más rápidos que el recorrido secuencial con 100 000 vectores, a cambio de perder algunos vecinos; el IVF permite controlar esa pérdida con `nprobe` y el HNSW da mejor recall con coseno. La galería de errores mostró además el límite de la representación: con SIFT y bolsa de palabras visuales, dos objetos lisos sobre fondo blanco se parecen aunque sean de tipos distintos.

Referencias

1. Garcia-Molina, H., Ullman, J. D. y Widom, J. *Database Systems: The Complete Book*. Pearson, 2.ª edición, 2008.

2. Ramakrishnan, R. y Gehrke, J. *Database Management Systems*. McGraw-Hill, 3.ª edición, 2003.

3. Fagin, R., Nievergelt, J., Pippenger, N. y Strong, H. R. Extendible hashing: a fast access method for dynamic files. *ACM Transactions on Database Systems*, 4(3), 1979.

4. Guttman, A. R-trees: a dynamic index structure for spatial searching. *Proceedings of ACM SIGMOD*, 1984.

5. PostgreSQL Global Development Group. Documentación de `EXPLAIN`. `https://www.postgresql.org/docs/current/sql-explain.html`

6. Collins, J. et al. ABO: Dataset and Benchmarks for Real-World 3D Object Understanding. *Proceedings of CVPR*, 2022. `https://amazon-berkeley-objects.s3.amazonaws.com/index.html`

7. Robertson, S. y Zaragoza, H. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 2009.

8. Sivic, J. y Zisserman, A. Video Google: A Text Retrieval Approach to Object Matching in Videos. *Proceedings of ICCV*, 2003.

9. Jégou, H., Douze, M. y Schmid, C. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1), 2011.

10. Malkov, Y. A. y Yashunin, D. A. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4), 2020.

11. Documentación de FAISS. `https://github.com/facebookresearch/faiss/wiki`